

**MIT WORLD PEACE UNIVERSITY, PUNE**

School of Computing

**MINI PROJECT REPORT**

Foundations of Programming — CSE10010

**Pointers in C: Powering Game Engines**

**Submitted By**

Hardik Mathur PRN: 1262250848

Neel Yetawedakar PRN: 1262251841

Piyush Bhati PRN: 1262253783

Malhar More PRN: 1262250895

Aditya Shinde PRN: 1262252501

**Project Guide**

Dr. Renuka Mane

(FOP Program Co-Ordinator)

Submission Date: \_\_\_\_\_

Academic Year: 2025–2026

# 1. Acknowledgment

We would like to sincerely thank Dr. Renuka Mane, our FOP Program Co-Ordinator, Department of Computer Science, MIT World Peace University, Pune, for her valuable advice, consistent support, and constructive feedback throughout this project. Her expertise was instrumental in shaping the direction and quality of our work.

We also thank the university and the Department of Computer Science for providing the necessary infrastructure and resources to complete this research.

Finally, we acknowledge the open-source programming community and the extensive documentation available for the C programming language, both of which served as valuable references during the development process.

## 2. ABSTRACT

Pointers are a part of C programming. They can be tough to figure out. This project is trying to make pointers easier to understand by showing how they work in life like in the game engines we use to play games. We are looking at the basics of pointers how memory works, how to use memory when we need it how to use pointers with structures and how to update data when it changes.

We are using examples from games to show how pointers help us manage memory and get to data quickly. We are also writing example programs that show how to use pointers to update objects when they change handle memory when we need it and make programs run quickly.

Our results show that pointers make programs run smoothly and let us use techniques that are necessary for making games and other complex systems. Pointers are really important, for making programs that work well. We are showing how they can be used to make things better.

# **TABLE OF CONTENTS**

1)Acknowledgement

2)Abstract

3)Literature Review

4)System Design

5)Implementation

6) Results and Discussion

7)Future Scope

8) References

### **3. Introduction**

For most beginners, learning pointers can feel like a sudden and confusing shift in how they interact with a computer. We learn that a variable is a simple box holding a value, like a health bar or a score. However, pointers make you stop focusing on the "thing" and start understanding the "address" of where that thing is located in one's computer's memory. In high-performance gaming, this address is what creates the magic. When a game needs to show a huge dragon or a busy city, it can't waste time creating millions of copies of those objects just to move them or change their colour. Pointers let the game engine reference these large files with tiny, fast shortcuts. All this is made easy with the help of presentation and poster.

### **4. Background**

Pointers allow us to access memory directly which makes it easier to work with data run programs faster and use memory efficiently. Game engines rely heavily on pointers to manage objects, textures, physics and real-time updates.

### **5. Objectives**

- \* To understand the basics of pointers
- \* To show how pointers are used in game engines
- \* To make a system that can handle objects in time using pointers
- \* To see how pointers can improve performance

## 6. LITERATURE REVIEW

We looked at research papers and programming resources. Found:

### 1. The "Manual Labor" of Memory

The research says that good memory management in C is crucial. The human reality is that in most modern languages, the computer acts like a personal assistant who cleans up after we are done using the memory. In C, we are the one responsible for cleaning. Using a pointer is like getting a key to a storage locker. If we lose the key or forget to lock the door when we finish, the system will run out of space and remove our entire program. Beginners struggle because they have never had to manage the "mess" their code creates.

### 2. The "Real-Time" Pressure of Game Engines

The research says that engine design needs pointers for real-time speed. The human reality is that it's like directing a live play where, 60 times a second, you must move the scenery, change the lights, and update the actors. If you had to rebuild the stage every time by copying data, the play would come to a halt. Pointers are like laser pointers. You simply point at the scenery that's already there and say, "Move that." Beginners get confused trying to track five different "laser pointers" all aiming at the same prop at once without causing a crash.

### 3. The "Unpredictability" of Dynamic Systems

The research says that dynamic memory is flexible but difficult to debug. The human reality is that games are unpredictable. You don't know if a player will spawn one enemy or a thousand. To deal with this, you use the "Heap"—which is just a big, messy pile of spare parts. Pointers are the only way to find what you need in that pile. The problem here is the disappointment of crashing: you try to grab a part using an old address, realize someone else has already moved it, and suddenly the whole game disappears to the desktop. This can be very discouraging for beginners.

## 6.1 Approach

Our group made a very descriptive presentation and also a poster on how the pointers are used in game engines and what all modules and functions of C are used in the same . We tried to keep it simple so that everyone having basic idea of C and its functions can understand the topic very well . Later on , we also added a gameplay video on how pointers affect during dynamic gameplay referring the structure of a player with the help of pointers .

## 6.2 Modules

- **Pointer Basics Module:** This module teaches you to stop focusing on the "value" and start looking at the "map" of where data is stored in the hardware.
- **Structure Pointer Module:** This module shows you how to manage big, complex "folders" of data (like a Player) without the delay of moving them around.
- **Dynamic Memory Module:** This is the "Janitor" training where you learn to manually rent and return space in the computer's memory as needed.
- **Real-time Update Module:** This is the "Secret Sauce" for 60 FPS, allowing multiple game systems to observe and change the same object instantly.

## 6.3 The Techniques

- Dereferencing : It is the action of accessing or manipulating the **value** stored at the memory address the pointer holds, using the unary asterisk (\*) operator. The pointer itself stores the *address*, not the actual value.
- Call by Reference : It means passing the memory address of a variable to a function using pointers. This allows the function to directly access and modify the original variable, unlike "call by value" where only a copy is passed.
- malloc( ) : This command lets you take a specific amount of raw memory from the computer while the game is running.
- Structure Pointers : It is a pointer variable that stores the memory address of a structure variable. It allows you to access and manipulate the members of the structure indirectly, which is crucial for efficient memory usage and building complex data structures like linked lists.

## 7. IMPLEMENTATION

- Development Environment Language: C
- Compiler: Online C Compiler / GCC

We worked on this project using the C programming language. We chose C because it is efficient and lets me access memory directly using pointers. We Tested the programs using an Online C Compiler and the GCC compiler. These tools made it easy to debug execute and verify pointer operations. Using a compiler ensured that the code follows proper syntax and works the same on different systems.

#### Sample Concept Implemented :

In this project we implemented pointer concepts through practical examples inspired by game engine behaviour. One main implementation involved updating the players position using pointers .We directly modified the players coordinates, such as x and y positions using pointer variables. This showed real-time updates and efficient memory usage.

- \* We also demonstrated dynamic enemy creation using malloc().
- \* We allocated memory for enemy objects during runtime.
- \* This allowed efficient memory management.
- \* It simulates how game engines create objects dynamically based on gameplay requirements.
- \* It also helps avoid memory usage when objects are not needed.

We also implemented score tracking using pointer variables. We passed the score variable as a pointer to functions. Updates were made directly to the memory location. This approach reflects real-world scenarios where game scores or player statistics need to be updated without duplication of data.

Overall these implementations highlight how pointers enable data handling, dynamic memory allocation and real-time updates. These



are features, in modern game development systems that use C programming language and pointers.

## **CONCLUSION**

This project shows how pointers are used to make real-time systems . We also got an idea of how some features of C are used and are very helpful while memory allocation and this memory allocation is very important in game engine systems while developing games and even during dynamic game runtime . By using examples from games we made complex programming ideas easier to understand. We achieved our goals of understanding , implementing and analyzing pointer usage in games.

## **FUTURE SCOPE**

- Make a game using pointers
- Add graphics to the game
- Use pointers, with linked structures
- Study how to optimize memory in game engines